GraphRAG over Cyber Threat Reports: Grounding Language-Model Answers in a Neo4j Knowledge Graph of AnnoCTR and MITRE ATT&CK

Shane Waldrop
Team CTRL ALT DEFEAT • CS 6338 Knowledge Engineering
Angelo State University, San Angelo, Texas, USA
swaldrop2@angelo.edu

***Abstract*—Cyber threat intelligence (CTI) analysts ask questions joining what a threat report says with what MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) records, such as which mitigations address the most techniques a report describes. Retrieval-augmented generation (RAG) over report text alone cannot answer these multi-hop questions: the connecting facts are not in the text. This paper presents a GraphRAG system that joins ATT&CK v20.0 (1,785 entries, 20,426 relationships), loaded through a Resource Description Framework (RDF) graph, to 50 expert-annotated AnnoCTR reports in a Neo4j labeled property graph. A large language model (LLM) extracts technique, tactic, group and software mentions under an enforced schema, and a linker resolves them to ATT&CK by exact name, then by embedding similarity. Mention edges store the evidence sentence, link method, confidence, model and timestamp. Answers draw on report chunks plus graph facts ranked by personalized PageRank; the prompt requires a citation after every statement, and code checks every cited identifier against the context. On 34 held-out test reports, extraction and linking reach a parent-level F1 of 0.709, against 0.442 for exact name matching. On 24 multi-hop questions, GraphRAG names the reference ATT&CK identifiers with a mean F1 of 0.492, against 0.125 for a text-only baseline, and is rated more complete (3.50 against 1.38 of 5) but less factual (3.17 against 4.63): the baseline mostly declines, while GraphRAG's wrong items follow errors in its own graph (inference). Unsupported statements are rare in both systems (under 2%), and all 229 GraphRAG citations point into its context.**

***Index Terms*—GraphRAG, knowledge graphs, cyber threat intelligence, MITRE ATT&CK, entity linking, Neo4j, personalized PageRank**

I. Introduction

Problem. A security operations analyst who reads a vendor threat report has to turn its narrative into decisions: which adversary techniques it describes, which threat groups are known to use them, and which mitigations would block the most of them. The report supplies one half of each answer; MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) [2], a curated public knowledge base of tactics, techniques, groups, software and mitigations, supplies the other. The question "Which ATT&CK mitigations address the largest number of the techniques described in report X?" takes four steps: from the report to the techniques it describes, from each sub-technique to its parent, from each technique to its mitigations, and a count. Retrieval-augmented generation (RAG) [3], which retrieves report passages and hands them to a large language model (LLM), finds the right report but not the ATT&CK relationships. In the evaluation below, a text-only RAG baseline declined 21 of 24 such questions, stating that its context lacked the ATT&CK information.

Domain. Cyber threat intelligence (CTI) suits a knowledge-graph approach for three reasons. Claims must be verifiable, because analysts act on them, so every answer needs provenance. Both kinds of source exist in public: AnnoCTR [1] annotates 400 threat reports donated by five vendors (Intel 471, Lab52 of S2 Grupo, Proofpoint, QuoIntelligence and Zscaler) and, in 120 of them, links mentions to ATT&CK; these expert links serve as gold labels (license: Creative Commons Attribution-ShareAlike, CC BY-SA 4.0); ATT&CK provides the structured side as a versioned STIX (Structured Threat Information Expression) file. And the questions are multi-hop by nature, which is where graph retrieval is expected to help [5], [6].

Contributions. The system, built and evaluated for this project, makes five design choices:

- A two-source knowledge graph: ATT&CK v20.0 enters through an RDF/OWL (Resource Description Framework / Web Ontology Language) graph built with rdflib and checked by a SPARQL (SPARQL Protocol and RDF Query Language) count of its classes, then becomes a Neo4j labeled property graph (LPG), joined to report text by extracted and linked mentions whose edges carry full provenance.
- Schema-enforced LLM extraction and named entity disambiguation (NED) whose threshold was calibrated on a development split before a single run on the test split.
- Report-anchored GraphRAG, that is, retrieval-augmented generation from a graph [4], [6]: personalized PageRank [7] seeded on the nodes a report mentions, summary counts computed in Cypher, and an automatic check that every cited identifier was in the model's context.
- Graph analytics with Neo4j Graph Data Science (GDS): PageRank [8], Louvain communities [9] and FastRP (Fast Random Projection) embeddings [10], with a topological analysis of the ATT&CK graph.
- A reproducible evaluation: every model reply is saved under the SHA-256 (Secure Hash Algorithm, 256-bit) hash of its request, so the results can be recomputed

TABLE I
RELATIONSHIP TYPES, COUNTS AND PROPERTIES

Type: from → to (count)	Properties besides `edge_id`, `source`
HAS_CHUNK: Report → Chunk (149)	none
MENTIONS: Chunk → ATT&CK node (1,880) or Entity (918)	report_id, chunk_id, mention_text, evidence, category, attack_name, link_method, confidence, model_confidence, model, reply_key, extracted
USES: Software → Technique (11,608), Group → Technique (4,815), Group → Software (1,166); from text (299)	ATT&CK: stix_id, description; text: report_id, chunk_id, evidence, reply_key
SUBTECHNIQUE_OF: Technique → Technique (479)	stix_id
PART_OF_TACTIC: Technique → Tactic (881)	none (from kill-chain phases)
MITIGATES: Mitigation → Technique (1,477)	stix_id, description

from the repository without a new model call, and the scripts behind this paper's tables and figures are in its `docs/final` folder; the comparison uses a text-only baseline, blind human ratings, RAGAS (Retrieval Augmented Generation Assessment) [11] and 95% bootstrap intervals [12].

This introduction and Sections II–VI follow the six parts the assignment names: schema and ontology, extraction and ingestion, graph analytics and reasoning, LLM integration, and evaluation. All results are *measured* on the committed data unless marked otherwise; code, data, saved replies and results are in the public repository https://github.com/Wittyusername12/ctrl-alt-defeat-graphrag (branch `build`).

II. KNOWLEDGE GRAPH SCHEMA AND ONTOLOGY DESIGN

A. Node labels, relationship types and edge properties

The graph has eight node labels (Fig. 1). Report and Chunk hold the text. Technique (sub-techniques included), Tactic, Group, Software and Mitigation hold ATT&CK entries, keyed by their ATT&CK identifier (ID) in `attack_id`, with the properties `name`, `aliases`, `description`, `url` and `stix_id`, plus `is_subtechnique` on Technique, `software_type` (malware or tool) on Software and `shortname` on Tactic; the analytics of Section IV add `pagerank`, `community` and `fastrp`. Entity holds the name from a mention the linker left unlinked, either because no ATT&CK entry matched closely enough or because the model's own confidence fell below the cut-off c (Section III-D); it is keyed by category and normalized name, so repeated names merge into one node. Six relationship types connect them (Table I). Every relationship carries `edge_id`, the identifier an answer cites, and `source`, either `attack` (recorded by MITRE) or `text` (extracted from a report). The loaded graph holds 2,258 nodes: 1,785 ATT&CK entries, 50 reports (16 dev, 34 test), 149 chunks and 274 Entity nodes.

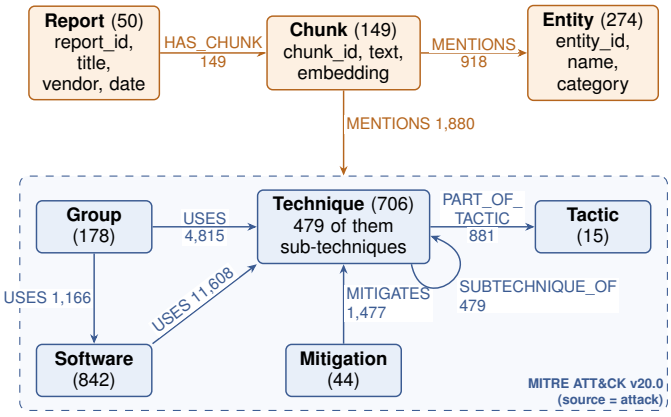


Fig. 1. Knowledge-graph schema with node and relationship counts. Orange: report text and its derived edges (`source = text`); blue: ATT&CK (`source = attack`). MENTIONS reaches any ATT&CK node or an Entity. Besides the 17,589 ATT&CK USES relationships, 299 USES edges extracted from text join the nodes that a report's group, software and technique mentions link to.

B. Provenance, timestamps and confidence

Each text-derived MENTIONS edge records where the fact came from and how it was resolved: the report and chunk; the mention text and the whole evidence sentence, both copied by the model and checked by code to occur in the chunk (with quotes, dashes and white space normalized); the category and the ATT&CK name the model proposed; the link method (*exact*; *exact, tie broken by category*; *exact, tie broken by the parent technique*; *embedding*; or *unlinked*, with its reason); the link confidence, present only for a link, which is 1.0 for an exact name match and the cosine similarity for an embedding match, never an invented constant; the model's own confidence; the model name; `reply_key`, the SHA-256 hash of the extraction request whose saved reply produced the edge; and `extracted`, a Coordinated Universal Time (UTC) timestamp. ATT&CK's USES and MITIGATES edges keep MITRE's STIX identifier and description, and 18,069 of the 19,061 descriptions carry MITRE's own source citations (5 USES relationships have no description); SUBTECHNIQUE_OF edges keep the STIX identifier only, and PART_OF_TACTIC edges, derived from kill-chain phases, have neither. A cited chunk resolves by its `chunk_id` and a cited ATT&CK edge by its `edge_id`; one Cypher query turns a cited mention back into its evidence:

```
CYPHER 25
MATCH (c:Chunk)-[m:MENTIONS {edge_id: $cited}]->(n)
RETURN c.chunk_id, m.mention_text, m.evidence,
       m.link_method, m.confidence, m.model,
       m.reply_key, m.extracted, n.attack_id
```

C. Ontology mapping: RDF/OWL versus LPG

ATT&CK v20.0 is published as a STIX 2.0 JavaScript Object Notation (JSON) bundle (26,689 objects). The loader (`load_attack.py`) keeps the live objects (neither revoked nor deprecated) of six STIX types, mapped to five OWL classes: `attack-pattern` to Technique, `x-mitre-tactic` to Tactic, `intrusion-set`

```
<.../groups/G0099> cad:uses <.../software/S0385> .
<.../edge/attack:USES:G0099:S0385> a rdf:Statement ;
  rdf:subject <.../groups/G0099> ;
  rdf:predicate cad:uses ;
  rdf:object <.../software/S0385> ;
  cad:edgeId "attack:USES:G0099:S0385" ;
  cad:source "attack" ;
  cad:stixId "relationship--00c26d3c-..." ;
  cad:description "APT-C-36 has used a customized ..." .

(:Group {attack_id: 'G0099'})
  -[:USES {edge_id: 'attack:USES:G0099:S0385',
           source: 'attack',
           stix_id: 'relationship--00c26d3c-...',
           description: 'APT-C-36 has used a ...'}]->
(:Software {attack_id: 'S0385'})
```

Listing 1. The fact "APT-C-36 uses njRAT" as RDF (Turtle; `.../` abridges https://attack.mitre.org/ and the project's namespace, `cad:` is the project ontology) and as an LPG relationship (Cypher pattern). The STIX identifier and the description, which also holds Markdown links and MITRE's citations, are abridged.

to Group, `malware` and `tool` to Software, and `course-of-action` to Mitigation. Their `uses`, `mitigates` and `subtechnique-of` relationships become OWL object properties, with `rdfs:domain` and `rdfs:range` where a property has a single one (USES has none, since groups use techniques and software while software uses techniques), and PART_OF_TACTIC comes from each technique's kill-chain phases. Each object becomes an RDF resource named by its attack.mitre.org address [13], [14]. A SPARQL query counts the resources in each class and Python counts the `rdf:Statement` resources of each relationship kind; the loader compares both with the STIX file, writes the same nodes and relationships to Neo4j, and reads them back: 1,785 nodes and 20,426 relationships at every stage. The ontology is the project's own (prefix `cad:`): its classes mirror the STIX object types it loads, and it is not aligned with an external ontology such as the Unified Cyber Ontology (UCO). Listing 1 shows one relationship in both models.

RDF 1.1 cannot attach values to a triple, so the relationship's identifier, source, STIX identifier and description need a reified `rdf:Statement` resource. In the ATT&CK graph, 161,162 of the 194,611 triples (82.8%) describe the 20,426 statements, beside the 20,426 plain relationship triples; the LPG stores the same information as 20,426 relationships with properties. Three needs decided the storage model. First, provenance on every text-derived edge (up to twelve values per MENTIONS edge besides its identifier and source) would need reification or RDF-star in RDF. Second, GDS algorithms and vector indexes run in the same store as the graph. Third, the multi-hop questions are Cypher path patterns. RDF/OWL remains the interchange and checking layer: the Turtle file (15.8 MB) is a standards-based copy of the ATT&CK ontology, the SPARQL counts check the load, and OWL's classes and properties survive as labels and relationship types. The one rule this design needs, the sub-technique hierarchy (Section IV-E), runs as Cypher at query time instead of in an OWL reasoner. Neo4j's RDF plugin, n10s (neosemantics), was not used: Neo4j Desktop does not offer it, and no n10s release targeted Neo4j 2026.x when checked on 19 October 2026.

D. Constraints and indexes

Eight uniqueness constraints, one key per label, back every `MERGE` with an index. A vector index on `Chunk.embedding` (1,536 dimensions, cosine, no quantization; a hierarchical navigable small world, HNSW, index [15]) serves chunk search, and three vector indexes on FastRP vectors (128 dimensions, for Technique, Group and Software) serve the chat page's nearest-neighbor lists.

III. Extraction and Ingestion Architecture

Fig. 2 shows the pipeline. Report text and ATT&CK enter on separate paths and meet in Neo4j through the entity resolver.

A. Data and splits

AnnoCTR (commit `d510b69`) splits its 120 reports into train (70), dev (16) and test (34). The gold standard, written by `gold.py` from the experts' links and mapped to ATT&CK v20.0, holds 213 dev and 469 test (report, ATT&CK entry) facts; the test facts are 327 techniques, 85 tactics, 13 groups and 44 software. Ten links to the revoked T1562 become T1685, and six links to ATT&CK Mobile IDs are left out. The experts labeled parent techniques only: none of the 682 facts names a sub-technique. Train reports supply the prompt's two worked examples, dev reports set the linking threshold, and the test reports were extracted, linked and scored once. Linking cannot rely on names: only 288 of the 1,526 test mentions with an expert link have a text equal to an ATT&CK name or alias, and 45.3% describe a behavior without naming it.

B. Text ingestion and normalization

`annoctr.py` removes embedded base64 images (keeping their alt text) and cuts each report into 4,000-character chunks that overlap by 400 characters (dev 50, test 99 chunks), so any sentence of up to 400 characters that crosses a boundary is whole in one chunk. All text comparisons map typographic quotes and dashes to plain ones and collapse white space. On the ATT&CK side, the loader drops revoked and deprecated objects (157 revoked-by mappings exist in v20.0), uses the version-20 names (the former Defense Evasion tactic is now Stealth and Defense Impairment), and checks the ATT&CK file's SHA-256 hash before reading it; the setup check compares all 124 data files with a committed list of hashes.

C. LLM extraction: prompt, temperature and schema enforcement

Each extraction request holds a system prompt, two worked examples (the openings of two train reports, with hand-written replies of 12 and 16 mentions) and the chunk. The prompt asks for one entry per place where the chunk names or describes a technique, tactic, group or software: the words copied exactly, the category, the ATT&CK v20 name it denotes (a sub-technique as "Parent: Sub-technique"), the whole evidence sentence and a confidence; then the "uses" relations between entries, referenced by local entry IDs (m1, m2, ...). Code

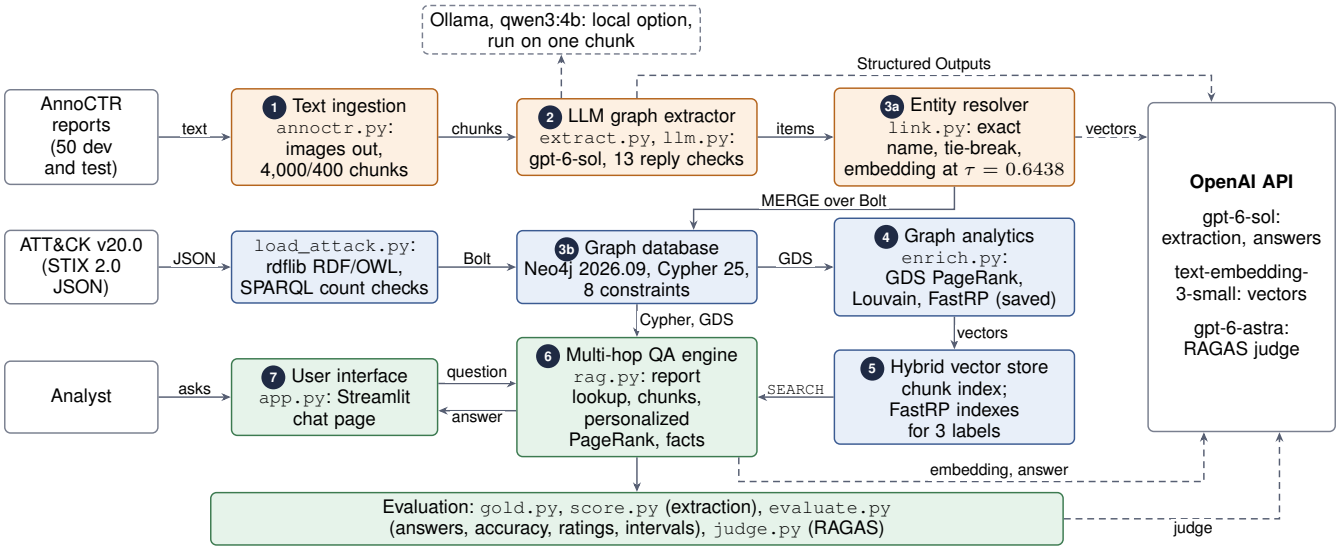


Fig. 2. Architecture as built. Numbered boxes are the seven subsystems of the design: 1 text ingestion, 2 LLM graph extractor, 3a entity resolver, 3b graph database, 4 graph analytics, 5 hybrid vector store, 6 multi-hop question-answering (QA) engine, 7 user interface. Dashed arrows are calls to model services: the OpenAI application programming interface (API), reached over HTTPS (Hypertext Transfer Protocol Secure), whose replies are saved and reused, and Ollama, a local option for the extractor that was run on one chunk (Section III-C). Bolt is Neo4j's network protocol.

builds every permanent identifier from the chunk's identifier and the model's local entry ID (Fig. 4 cites `...#0/m18`). An excerpt of the prompt, with cuts marked [...]:

```
[...]
List one entry for each place where the chunk names or
    ↪ describes a concept:
[...]
- text: the words in the chunk that name or describe the
    ↪ concept, copied exactly
[...]
- attack_name: the concept's name in ATT&CK Enterprise
    ↪ version 20, spelled as ATT&CK spells it; for a sub-
    ↪ technique write "Parent technique: Sub-technique";
    ↪ write "" when no ATT&CK entry fits
- evidence: the whole sentence of the chunk that contains
    ↪ the text, copied exactly
[...]
Copy text and evidence character for character from the
    ↪ chunk. Every entry must rest on the chunk's own
    ↪ words; use your knowledge of ATT&CK only to choose
    ↪ attack_name. [...]
```

The model is gpt-6-sol through OpenAI's Chat Completions application programming interface (API) with reasoning effort *none*, temperature 0, seed 7 and at most 4,000 completion tokens; a reply cut off at that cap is requested once more with 8,000. Schema enforcement has two layers. Pydantic models compile to OpenAI Structured Outputs, a JSON schema the reply must follow; code then checks 13 drop reasons, each with its own counter: refusal, content filter, cut-off, invalid JSON, a repeated or empty entry, a text or evidence sentence not found in the chunk, a text not inside its evidence, a relation to a missing entry, a relation pointing the wrong way, categories that cannot use each other, and relation evidence not in the chunk. Separately, every reply is saved under the SHA-256 hash of its request and replayed on reruns.

On the dev split all 50 replies were complete (980 mentions and 121 relations kept, $1.13 at list price). On the test split 98 of 99 replies were complete; the one failure was a chunk holding a Markdown table of techniques, whose cells the model copied as evidence sentences until the reply hit both caps. The test run kept 1,818 mentions (962 technique, 138 tactic, 207 group, 511 software) and 178 relations, dropped 9 mentions (7 whose evidence sentence was not in the chunk) and 2 relations from complete replies, and used 389,031 input and 147,902 output tokens ($2.25, 1,047 s, 10.3 s per reply on average). After the full runs, the same prompt ran once through Ollama with qwen3:4b on the project laptop's central processing unit (CPU; an Intel i7-1255U, with no usable graphics processing unit): the reply followed the schema but took 838.3 s, about 100 times the 8.51 s of gpt-6-sol on the same chunk, and only 1 of its 8 kept mentions named an ATT&CK entry exactly. The local path works, but at that speed the 149 dev and test chunks would take about 35 hours on this laptop (estimated).

D. Named entity disambiguation

`link.py` resolves each kept mention in four steps. (1) Normalize the names. (2) Look up the model's ATT&CK name, then the mention text, among 2,927 keys (2,448 distinct ATT&CK names and aliases plus 479 "Parent: Sub-technique" names) across all five ATT&CK labels, never only the label the model guessed. (3) Settle a key shared by several entries (49 keys; "Carbanak" names a group and a piece of software) by the model's category and then by the named parent, never by an arbitrary pick. (4) Otherwise embed the model's name, the mention text and its evidence sentence with text-embedding-3-small and take the most similar ATT&CK entry, compared as "name: description". A link is kept when it is exact or its cosine similarity reaches τ, and the model's own confidence reaches c; any other mention becomes an Entity node. The pair (τ, c) was chosen on the dev reports from 4,554 candidates by

TABLE II
TOPOLOGY OF THE ATT&CK PROJECTION (1,785 NODES)

Measure	Value
Relationships (distinct node pairs; no parallels, no loops)	20,426
Density (pairs / possible pairs)	0.0128
Degree: mean; median; maximum (T1105)	22.9; 12; 504
Mean degree: Technique; Tactic; Group; Software; Mitigation	28.0; 58.7; 33.6; 15.2; 33.6
Connected components	1
Average shortest path; diameter; radius (hops)	2.76; 6; 4
Ordered node pairs within 3 hops	88.1%
Average local clustering coefficient	0.100
Louvain: communities; modularity; levels	6; 0.2953; 3
Spearman correlation of PageRank and degree	0.983

the highest parent-level F1 (the harmonic mean of precision and recall against the gold facts, a sub-technique counting as its parent; Section VI): $\tau = 0.6438$ and $c = 0.75$ raised dev F1 from 0.591 (at $\tau = 0.40$ with no cut-off) to 0.687, and both values were fixed before the test split was opened. On the test reports the linker made 1,169 exact links, 1 exact link with a tie broken by category and 43 embedding links, and left 605 mentions (33%) as Entity nodes: 307 whose closest entry fell below τ and 298 whose model confidence fell below c, 250 of which had an exact-name match.

E. Loading

Writes use Cypher 25 `MERGE` in batches of 1,000, so a rerun changes nothing; reloading a report first removes its text-derived nodes and relationships, and every load is read back and counted (test: 34 of 34 reports, 99 chunks, 1,818 MENTIONS and 178 text USES edges).

IV. GRAPH DATA SCIENCE AND REASONING

A. Projection and topology

GDS 2026.09 runs on one undirected Cypher projection of the ATT&CK part of the graph: the five ATT&CK labels and the four relationship types as MITRE records them (`source = attack`), so that loading reports cannot change the results. GDS reports 1,785 nodes and 40,852 relationships (each of the 20,426 counted in both directions). Table II gives the topology, computed in Python from the same projection; its node and relationship counts and its Louvain modularity equal the GDS output. The graph is one connected component with a heavy-tailed degree distribution (Fig. 3): the median entry has 12 relationships, while T1105 Ingress Tool Transfer has 504, of which 411 are USES from software and 90 USES from groups. No two entries are more than 6 hops apart, 88.1% of ordered pairs are within 3 hops, and the 2-hop neighborhood of a single entry holds 623 entries on average (35% of the graph), so retrieval ranks the neighborhood of a report's entities instead of taking it whole (Section V-A).

B. Centrality

Global PageRank (damping 0.85, one thread) ranks T1105 Ingress Tool Transfer (18.28), T1071.001 Web Protocols (14.98), T1082 System Information Discovery (14.83), T1059.003 Windows Command Shell (13.42) and T1083 File and Directory Discovery (13.18) highest. On this undirected graph PageRank follows degree closely (Spearman 0.983, with tied ranks averaged), so it measures how widely an entry is used more than any separate notion of authority (inference). The highest-ranked groups are Kimsuky, APT28 and APT29; software, Cobalt Strike, Empire and Mimikatz; mitigations, M1018 User Account Management, M1056 Pre-compromise and M1047 Audit. The scores, community numbers and FastRP vectors are saved to files and written to the ATT&CK nodes on every machine; the chat page searches the FastRP vectors for similar entries, and retrieval uses the personalized variant of PageRank (Section V-A).

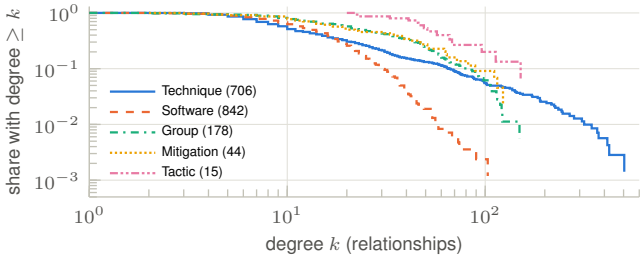


Fig. 3. Share of each label's entries with at least k relationships in the ATT&CK projection (log scales). Techniques hold both the most low-degree entries (52.8% have at most 10 relationships, against 42.5% of software) and the hubs (38 have 100 or more, up to 504); software's tail is short (1.5% have 50 or more; maximum 103).

TABLE III
LOUVAIN COMMUNITIES (TE: TECHNIQUES, TA: TACTICS, G: GROUPS, S: SOFTWARE, M: MITIGATIONS)

#	Size	Te/Ta/G/S/M	Top tactics of its techniques; highest-PageRank technique, group, software
1	560	76/0/42/442/0	Stealth, Discovery; T1105, Gamaredon Group, InvisiMole
2	481	361/9/12/63/36	Persistence, Stealth; T1036, APT29, Mini Shai-Hulud
3	228	66/2/73/85/2	Discovery, Lateral Movement; T1059.001, APT41, Cobalt Strike
4	228	63/1/13/147/4	Command and Control, Stealth; T1140, Lazarus Group, Emotet
5	156	100/2/37/16/1	Resource Development, Reconnaissance; T1566.002, Kimsuky, PoisonIvy
6	132	40/1/1/89/1	Impact, then Defense Impairment and Stealth (tied); T1112, Water Galura, DarkGate

C. Communities

Louvain (GDS defaults, one thread; the result is saved to a file) found 6 communities with modularity 0.2953 in 3 levels. Table III lists their make-up. The communities follow kinds of entry and stages of an attack: malware with the techniques it implements; host techniques together with 36 of the 44 mitigations; hands-on intrusion groups with dual-use tools and lateral movement; command-and-control malware; pre-compromise reconnaissance and resource development; and impact. These readings are interpretations of the make-up, not measurements.

D. Graph embeddings

FastRP [10] embeds each entry in 128 numbers from its 2- and 3-hop neighborhood (iteration weights 0, 1, 1; no self-influence; seed 42; one thread). Its random start depends on internal node identifiers, so the vectors are computed once and saved. The nearest group to G0129 Mustang Panda is G1014 LuminousMoth (Neo4j similarity 0.948, cosine 0.897), then LazyScripter (0.929) and Earth Lusca (0.928). The closeness comes from shared USES edges: 20 of LuminousMoth's 30 USES targets are also Mustang Panda's. ATT&CK records a link between the two groups only in text (LuminousMoth's description reports that some researchers connect it with Mustang Panda, and Mustang Panda's aliases include LUMINOUS MOTH), not as an edge, so this is one example of the embedding placing related groups together, not a new finding.

E. Path calculations and rule-based reasoning

Multi-hop paths. Each benchmark question (Section VI) stores a Cypher query over its path pattern, run with the report's gold entries as parameters; all 24 return the reference answers in Neo4j. The query for the mitigation questions counts, for each mitigation, the report's techniques (`$techniques`; for Q01, the 13 the experts labeled) that it addresses directly or through a sub-technique, and returns the mitigations with the highest count. Reformatted (the stored query starts with `CYPHER 25` and quotes every name in backticks):

```
MATCH (m:Mitigation)-[:MITIGATES]->(:Technique)-[:
    ↪ SUBTECHNIQUE_OF*0..1]->(t:Technique)
WHERE t.attack_id IN $techniques
WITH m, collect(DISTINCT t.attack_id) AS addressed
WITH max(size(addressed)) AS most, collect({id: m.attack_id
    ↪ , addressed: addressed}) AS rows
UNWIND rows AS row WITH row, most WHERE size(row.addressed)
    ↪  = most
RETURN row.id AS answer, row.addressed AS support ORDER BY
    ↪ answer
```

Hierarchy rule. A relationship ATT&CK records for a sub-technique also holds for its parent: $USES(x, s) \wedge SUBTECHNIQUE_OF(s, t) \Rightarrow USES(x, t)$, and likewise for MITIGATES. The rule matters because 8,877 of ATT&CK's 16,423 USES relationships to techniques point at sub-techniques, while the experts label parents only. Queries apply it with the variable-length step `-[:SUBTECHNIQUE_OF*0..1]->`, and the scorer and the summary lines apply it at parent level. The rule writes no relationship, so every derived fact still cites a recorded edge. **Personalized PageRank** is the path computation made for each question (Section V-A).

V. LLM Integration and GraphRAG Performance

A. Retrieval and prompt

`rag.py` answers a question in seven steps. (1) *Report lookup*: a quoted span equal to a report's normalized title anchors that report; all 24 benchmark questions anchored exactly the reports they name. (2) *Chunks*: the question's embedding selects up to 4 most similar chunks of the anchored reports by exact cosine (the chunk index serves questions that name no report). (3) *Seeds*: every node the anchored reports mention. (4) *Walk*: personalized PageRank [7] on an undirected projection of the whole graph (2,258 nodes; USES from ATT&CK and from text, SUBTECHNIQUE_OF, PART_OF_TACTIC, MITIGATES), damping 0.85, at most 40 iterations, one thread, with the seeds as source nodes; the 200 best nodes are kept. HippoRAG [5] uses a similar walk for multi-hop retrieval. (5) *Summary lines*: report lines list each anchored report's entries by kind; then Cypher counts, for every ATT&CK entry one relationship from those entries and each relationship type, how many of the entries it links to by ATT&CK relationships (a sub-technique link counting for its parent), with one edge ID per counted entry, so the model reads counts instead of computing them. At most 40 node lines are kept, taken in turn from MITIGATES, USES and PART_OF_TACTIC, highest count first; PageRank only breaks ties, so a line may describe a node outside the 200 kept (200 of the 960 node lines for the 24 questions did). (6) *Facts*: up to 40 mention facts and 60 relationship facts among the kept nodes, seed to seed first, then one and two steps out, taken in turn from each type. (7) *Answer*: gpt-6-sol (reasoning effort *none*, temperature 0, seed 7, at most 1,500 tokens) with the system prompt below and a user message holding the question, the chunks and, for GraphRAG only, the facts. Code parses every `[chunk:...]` and `[edge:...]` citation and flags any identifier that was not in the context (a citation-ID validity check: it shows that the cited item was supplied, not that it supports the claim). The text-only baseline runs steps 1, 2 and 7 with the same chunks, so the two systems differ only in the graph context.

```
You answer questions about cyber threat reports. Use only
    ↪ the context given after the question: report chunks
    ↪ , and knowledge-graph facts when there are any.
    ↪ After every statement, cite in square brackets the
    ↪ IDs of the chunks or facts that support it, exactly
    ↪  as they appear in the context, for example [chunk:
    ↪ report_2020-01-01_example#0] or [edge:attack:
    ↪ MITIGATES:M1040:T1055]. Do not use knowledge from
    ↪ outside the context. If the context does not answer
    ↪  the question, say what is missing.
```

B. An answer and its provenance

Fig. 4 shows question Q10 answered by both systems, with each GraphRAG citation resolved to its record. The baseline's chunks name APT-C-36 and njRAT but hold no ATT&CK relationship, so the baseline declines. GraphRAG answers from the ATT&CK edge `USES(G0099, S0385)` and the two mention edges that tie the report to the group and to njRAT. LimeRAT, which the report also names, has no ATT&CK entry; its mention became an Entity node (*unlinked*, best embedding match below τ), and the answer notes that the supplied ATT&CK relationship covers njRAT only. On the chat page (Fig. 2, box 7), each answer appears beside the baseline's with its citation check and the context the model saw, and GraphRAG's cited relationships and chunks are drawn as a graph, read from Neo4j by their edge IDs; the session's earlier questions and answers stay on the page.

Q10. Which malware or tools mentioned in Lab52's May 2021 report "Literature lover targeting Colombia with LimeRAT" does MITRE ATT&CK record as used by the threat group the report names?

GraphRAG (12,557 prompt tokens, 3.97 s): **njRAT (S0385)** is the malware mentioned in the report that MITRE ATT&CK records as used by **APT-C-36 (Blind Eagle)**. The report also mentions LimeRAT, but the provided ATT&CK use relationship is for njRAT. [edge:text:MENTIONS:...#0/m18] [edge:text:MENTIONS:...#1/m5] [edge:attack:USES:G0099:S0385] [edge:text:MENTIONS:...#1/m7]

Text-only baseline: The report names **APT-C-36 (Blind Eagle)** and mentions **LimeRAT** and **njRAT**, but the provided context contains no MITRE ATT&CK records showing which of those tools ATT&CK attributes to the group. [chunk:...#0][chunk:...#1]

Cited records. `attack:USES:G0099:S0385`: APT-C-36 (G0099) USES njRAT (S0385), STIX `relationship--00c26d3c-...`, "APT-C-36 has used a customized version of njRAT." `...#0/m18`: chunk 0 mentions "njRAT"; evidence "...loading a Delphi version of the njRAT in July 2020..."; link exact, confidence 1.0; gpt-6-sol, reply `0e06b741...`, extracted 2026-10-20T02:14:38Z. `...#1/m5`: "APT-C-36" linked exactly through the alias Blind Eagle. `...#1/m7`: "LimeRAT" → Entity `software:limerat`, unlinked.

Fig. 4. Question Q10: both answers (verbatim, with their Markdown bold shown as bold and the report identifier shortened to ...) and the records behind GraphRAG's four citations, as stored on the edges (the ATT&CK description as cleaned for the model).

TABLE IV
BENCHMARK QUESTION PATTERNS (24 QUESTIONS, 20 TEST REPORTS; 10 PATTERNS IN 8 ROWS, AS TWO ROWS EACH JOIN TWO RELATED PATTERNS)

Pattern	n	Path (Report → ...)
Top mitigations	5	MENTIONS Technique, hierarchy, ←MITIGATES Mitigation, counted
Tactics	4	MENTIONS Technique →PART_OF_TACTIC Tactic
Group's software (incl. inverse)	3	MENTIONS Group →USES Software ←MENTIONS Report
Group's / software's techniques	5	MENTIONS Group or Software →USES Technique, hierarchy, ←MENTIONS Report
Group using most techniques	1	as above, counted per group
Techniques a software lacks	1	MENTIONS Technique with no USES path from the software
Shared techniques	3	Report A →MENTIONS Technique ←MENTIONS Report B
Groups using every technique	2	MENTIONS Technique, hierarchy, ←USES Group, for all

C. Effect of the graph context

The assignment asks how graph context improves factual accuracy, reduces hallucination and provides provenance; over the 24 benchmark questions (details in Section VI), the measurements answer each in turn. *Factual accuracy*: the mean F1 of the ATT&CK IDs an answer names against the reference rises from 0.125 to 0.492 (+0.367, 95% interval [0.208, 0.530]), recall from 0.129 to 0.571, and blind-rated Completeness from 1.38 to 3.50 of 5; blind-rated Factuality, however, falls from 4.63 to 3.17, because GraphRAG states items, some of them wrong, where the baseline mostly states that it cannot answer. *Hallucination*, measured as answer statements the RAGAS judge found unsupported by the answer's own context, is rare in both systems (3 of 167 statements against 2 of 121; difference +0.001 $[-0.030, 0.032]$), so the graph does not measurably reduce it. *Provenance*: GraphRAG made 229 citations (222 counting each ID once per answer), every answer cited at least one item, and every cited identifier was in the context; each resolves to an edge or chunk as in Fig. 4. Split at sentence-final punctuation, 58 of the 68 sentences in GraphRAG's answers carry their own citation, and 7 of the other 10 introduce a list whose items are cited. The graph context thus changes what the model can say more than how carefully it says it (inference). The price is a larger context (median 13,553 against 2,642 prompt tokens), a mean of $0.0304 against $0.0069 per question at list price, and a median of 4.10 s against 2.19 s end to end.

VI. Evaluation and Discussion

A. Benchmark and protocol

Questions. `eval_queries.json` holds 24 multi-hop questions in 10 patterns about 20 test reports (Table IV), each naming its report by title. Answers (1 to 6 items) were computed in Python from the gold facts and ATT&CK v20.0, never from the system's graph; each was recomputed by its Cypher query in Neo4j, all 24 were recomputed by an independent reimplementation, and three were compared item by item with attack.mitre.org. Each question also stores a reference answer in words.

Measures. *Extraction and linking*: precision, recall and F1 $= 2TP/(2TP + FP + FN)$, with TP, FP and FN the true positives, false positives and false negatives, of (report, ATT&CK ID) facts against the gold, pooled over the 34 test reports, at sub-technique level and at parent level (a sub-technique counts as its parent, as the experts label parents), beside an exact-name baseline that links every ATT&CK name or alias of a technique, sub-technique, tactic, group or software entry (2,407 names) found in a report as a whole word, ignoring case. *Answer-ID accuracy*: per question, the precision, recall and F1 of the IDs of the asked kind that the answer names outside its citations, averaged over the questions. *Blind human ratings*: one rater, the team's only member, who built the system, scored all 48 answers from 1 to 5 for Factuality, Completeness (the share of reference items given) and Coherence against the reference answer and anchored descriptions of each score; each question's two answers appeared as A and B in an order drawn with a fixed seed, with citations replaced by neutral numbers and the cited texts listed in one format for both systems. *RAGAS* 0.4.3 [11]: Faithfulness, AnswerRelevancy and ContextRecall, judged by gpt-6-astra at reasoning effort *low* (288 judge calls, $14.17 at list price). *Hallucination rate*: the share of answer statements the faithfulness verdicts mark unsupported, pooled over statements. *Operations*: seconds per stage, tokens and cost per question. *Intervals*: 95% percentile bootstrap [12], 10,000 resamples with a fixed seed, resampling the 24 questions for answer measures (both systems on the same draws) and the 34 test reports for extraction.

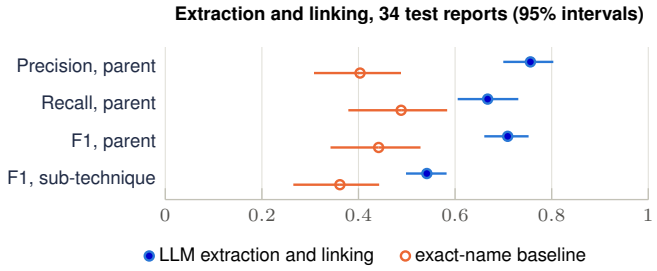


Fig. 5. Entity extraction and linking against the experts' 469 test facts.

B. Extraction and linking

At parent level the system reaches precision 0.756, recall 0.667 and F1 0.709 [0.661, 0.752], against 0.403, 0.488 and 0.442 [0.342, 0.528] for exact-name matching, a difference of +0.267 [0.172, 0.363] (Fig. 5). Most of the baseline's false links come from ATT&CK names that are everyday words: 300 of its 339 false facts come only from one-word names, and 143 only from "at", "malware", "server", "software" and "credentials", which link wrongly in 28, 27, 15, 14 and 13 reports. By kind, the system scores F1 0.894 on software, 0.813 on groups, 0.662 on tactics and 0.687 on techniques (baseline: 0.476, 0.647, 0.545 and 0.398); tactics and techniques score lowest, plausibly because many of their mentions are implicit (inference). At sub-technique level F1 is 0.542, since every sub-technique link counts against parent-only gold. In the report with the one cut-off chunk, 24 of the 48 gold facts are labeled only inside that chunk; over all reports, 21 of the 156 parent-level misses are labeled only inside chunks that yielded no mentions. For context only: the AnnoCTR paper's best document-level F1 when its systems choose the sentences themselves is 56.6 for techniques (a fine-tuned sentence classifier selects the sentences that describe a technique, then a generative linker links them) and 47.7 for tactics (the linker run on every sentence with a negative class), and 65.7 and 84.2 when the sentences the experts annotated are given [1, Table 9]. The settings differ (fine-tuned linkers against a prompted LLM; their technique and tactic scores against our parent-level facts), so the numbers do not rank the systems.

C. Answers

Table V and Fig. 6 give the results. Every measure of how much of the question an answer covers favors GraphRAG with an interval that excludes zero: answer-ID F1 and recall, Completeness, answer relevancy and context recall. Factuality favors the baseline, also with an interval that excludes zero. Faithfulness, the hallucination rate and Coherence do not separate the systems, and neither does the sum of the three human criteria (+0.54 [−0.38, 1.46]). The baseline's high Factuality comes from declining: it declined in 21 of its 24 answers, all but the three shared-technique questions, which the report texts can answer and which give most of its non-zero accuracy. All 21 were rated Completeness 1 and 19 of them

TABLE V
ANSWER RESULTS: MEANS OVER THE 24 QUESTIONS, EXCEPT AS NOTED FOR PRECISION AND THE HALLUCINATION RATE; DIFFERENCE GRAPHRAG − BASELINE WITH 95% INTERVAL

Measure	GraphRAG	Baseline	Difference
Answer-ID F1	0.492	0.125	+0.367 [0.208, 0.530]
Answer-ID recall	0.571	0.129	+0.442 [0.271, 0.617]
Answer-ID precision[a]	0.563	0.781	–
Factuality (1–5)	3.17	4.63	−1.46 [−2.08, −0.79]
Completeness (1–5)	3.50	1.38	+2.13 [1.50, 2.75]
Coherence (1–5)	4.88	5.00	−0.13 [−0.33, 0.00]
Sum of the three (3–15)	11.54	11.00	+0.54 [−0.38, 1.46]
RAGAS faithfulness	0.976	0.987	−0.011 [−0.045, 0.021]
RAGAS answer relevancy	0.737	0.169	+0.568 [0.442, 0.685]
RAGAS context recall	0.367	0.051	+0.316 [0.210, 0.428]
Hallucination rate[b]	0.018	0.017	+0.001 [−0.030, 0.032]

[a] Over the answers naming an ID of the asked kind: 21 GraphRAG, 4 baseline. [b] Unsupported statements, pooled: 3 of 167 and 2 of 121. Per question, GraphRAG wins, ties and loses 2/4/18 on Factuality, 18/6/0 on Completeness, 0/22/2 on Coherence and 13/1/10 on the sum.

TABLE VI
OPERATIONS PER QUESTION: MEDIAN (MAXIMUM), EXCEPT COST, A MEAN (TOTAL), AND CITATIONS, A TOTAL OVER THE 24 ANSWERS

	GraphRAG	Baseline
Lookup, chunks, seeds (s)	0.01, 0.25, 0.02	0.01, 0.02, –
Personalized PageRank; facts (s)	0.18; 0.26	–
Answer call (s)	3.33 (6.70)	2.16 (3.35)
End to end (s)	4.10 (8.15)	2.19 (3.38)
Prompt tokens	13,553 (22,595)	2,642 (4,744)
Cost at list price: mean (total, 24)	$0.0304 ($0.73)	$0.0069 ($0.17)
Citations, all in context (each ID once per answer)	229 (222)	66 (46)

Factuality 5, since a true statement that the context lacks the information is factual; RAGAS gave 19 of the 21 an answer relevancy of 0, the score for a noncommittal answer (Q11 and Q12, which name the report's groups and software before declining, scored 0.78 and 0.66).

Operations. The answer call dominates the time (median 3.33 s for GraphRAG); report lookup, chunk search, seeds, personalized PageRank (median 0.18 s) and the fact queries take a median of 0.77 s together, under a second for 20 of the 24 questions and at most 4.86 s (Q01, whose chunk stage took 3.14 s; Table VI). Both systems meet the design target of a median under 10 s. All OpenAI use for the project cost about $18.60 at list price: $3.43 for extraction (including an early trial), about $0.08 for development questions, $0.90 for the 48 benchmark answers and $14.17 for the RAGAS judge.

D. Error analysis

GraphRAG's answers follow its graph. In the five mitigation questions, every GraphRAG answer named the mitigations with the highest count in its own summary lines. Where the graph's counts put the reference on top (Q03, Q04, Q05, and M1040 in Q02), the answer contained it. In Q01 the system linked 14 parent techniques against the experts' 13, sharing 9: it added T1105, T1204, T1543, T1547 and T1555 and missed T1001, T1114, T1189 and T1606. The added techniques lift

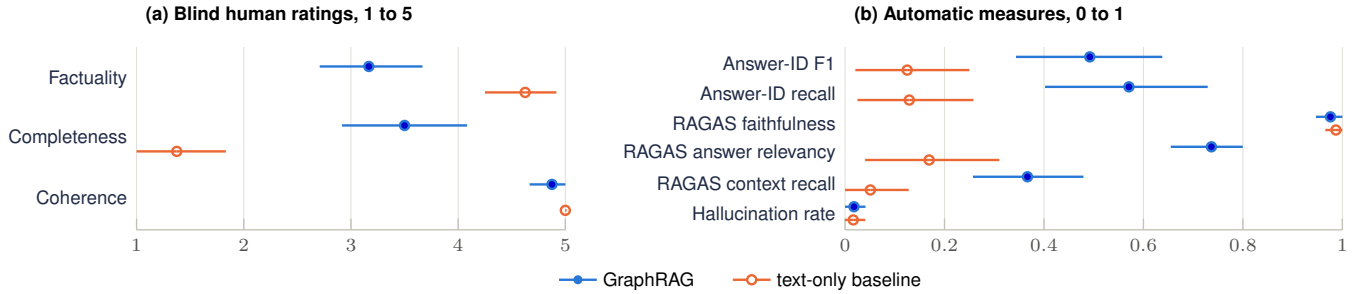


Fig. 6. Answer quality on the 24 benchmark questions: means with 95% bootstrap intervals.

M1018 and M1026 to 6, above the reference M1047 at 5, and the answer named M1018 and M1026. Three answers (Q02, Q04, Q05) also named mitigations below the top count, which costs precision. Some added links may be correct links the experts did not make: the gold holds one annotator's labels, and when the AnnoCTR authors had a second professional label techniques, the two sets agreed with an average F1 of 54%, a figure they say is not a proper agreement study and likely underestimates agreement [1].

Faithful but wrong. GraphRAG's statements are supported by its context (faithfulness 0.976) while its Factuality is lower, so its wrong items most likely come from the context itself, that is, from the graph the extraction built (parent-level precision 0.756), whose links the summary lines count (inference). Q01 is the example: the judge found every statement supported (faithfulness 1.00) while the reference answer is M1047.

Measure artifacts. The answer-ID measure credits listing: the baseline's Q14 answer says it cannot determine the techniques the group uses, then lists eight techniques from the report, three of them reference items, and earns recall 0.75, while the rater scored it Completeness 1. RAGAS's verdict step judges each statement without the question: GraphRAG's Q15 statement that its facts "support Cobalt Group (G0080) as an answer to the question" was marked unsupported for that reason alone.

E. Limitations

(1) One rater, who built the system, gives no agreement statistic, and the kind of evidence an answer cites can hint at its system. (2) 24 questions over 20 reports give wide intervals, and the patterns are unevenly represented. (3) The judge (gpt-6-astra) and the answering model (gpt-6-sol) both come from OpenAI's GPT-6 family (GPT: Generative Pre-trained Transformer), and judge models can favor text like their own [16]–[18]; both systems share them, which protects the comparison more than the absolute scores. (4) The gold holds one annotator's labels, so a correct link outside it counts as an error (in one dev report, the name APT27 links to G0027, whose ATT&CK aliases include it, while the expert did not link it to ATT&CK). (5) The "uses" relations extracted from text are loaded but not scored, because no expert labels exist for them. (6) The caps of 40 summary lines and 60 relationship facts can drop a needed line. (7) Repeatability rests on the saved replies; a new request at temperature 0 with a fixed seed is not guaranteed to return the same reply. (8) The local-model path was shown on one chunk only. (9) The citation check shows that cited identifiers were supplied, not that they support the statement, and 10 of GraphRAG's 68 answer sentences carry no citation of their own.

F. Future work

Extraction and linking quality is the lever the error analysis points to (interpretation). A fine-tuned sentence classifier that selects the sentences describing a technique before linking, the step behind AnnoCTR's best technique score without given sentences [1], could raise recall on implicit mentions, and table-aware chunking could recover the cut-off table chunk. Answers could commit to the Cypher-computed top count instead of restating counts in prose, which could remove the extra mitigations seen in Q02, Q04 and Q05. A larger benchmark with several raters would narrow the intervals, and expert labels for "uses" relations would let the extracted relations be scored.

VII. Conclusion

Joining threat reports to ATT&CK in one knowledge graph lets a language model answer multi-hop CTI questions that a text-only pipeline declines, with citations that trace its statements to report sentences and ATT&CK relationships through edge identifiers. The measured gains are in coverage: answer-ID F1 0.492 against 0.125 and Completeness 3.50 against 1.38. The measured cost is lower rated Factuality (3.17 against 4.63) than a baseline that mostly abstains, and the error analysis indicates that the answers follow the graph's extraction and linking (parent-level F1 0.709), mistakes included (inference). Faithfulness to the context is high (0.98 and 0.99) and unsupported statements are rare (under 2%) in both systems, with no measurable difference.

References

[1] L. Lange, M. Müller, G. Haratinezhad Torbati, D. Milchevski, P. Grau, S. Pujari, and A. Friedrich, "AnnoCTR: A dataset for detecting and linking entities, tactics, and techniques in cyber threat reports," in *Proc. LREC-COLING 2024*, Torino, Italy, 2024, pp. 1147–1160.

[2] B. Strom, A. Applebaum, D. Miller, K. Nickels, A. Pennington, and C. Thomas, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, Tech. Rep., Mar. 2020.

[3] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33*, 2020, pp. 9459–9474.
[4] D. Edge *et al.*, "From local to global: A graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024.
[5] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems 37*, 2024.
[6] B. Peng *et al.*, "Graph retrieval-augmented generation: A survey," arXiv:2408.08921, 2024.
[7] T. H. Haveliwala, "Topic-sensitive PageRank," in *Proc. 11th Int. World Wide Web Conf. (WWW)*, Honolulu, HI, USA, 2002.
[8] S. Brin and L. Page, "The anatomy of a large-scale hypertextual web search engine," *Computer Networks*, vol. 30, pp. 107–117, 1998.
[9] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech.*, vol. 2008, P10008, 2008.
[10] H. Chen, S. F. Sultan, Y. Tian, M. Chen, and S. Skiena, "Fast and accurate network embeddings via very sparse random projection," in *Proc. 28th ACM CIKM*, 2019.
[11] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. EACL 2024: System Demonstrations*, 2024, pp. 150–158.
[12] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.
[13] R. Cyganiak, D. Wood, and M. Lanthaler, Eds., "RDF 1.1 concepts and abstract syntax," W3C Recommendation, Feb. 2014.
[14] W3C OWL Working Group, "OWL 2 Web Ontology Language document overview (second edition)," W3C Recommendation, Dec. 2012.
[15] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
[16] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.
[17] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in *Advances in Neural Information Processing Systems 37*, 2024.
[18] K. Wataoka, T. Takahashi, and R. Ri, "Self-preference bias in LLM-as-a-judge," arXiv:2410.21819, 2024.